Install & Import Libraries

```
!pip install tensorflow opencv-python pillow



import os
import cv2
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras import layers, models
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorfl
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/loca
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.2.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.10.0)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.15.1)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensor
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.18.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tens
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tens
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorf
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensor
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorf
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboa
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->te
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->kera
```

Import Dataset

```
from google.colab import files
upload = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Unzip Image Dataset (FIXED)

```
!unzip img.zip -d /content/img
```

```
  inflating: /content/img/img/IMG_20241006_144505 - Copy.jpg
  inflating: /content/img/img/IMG_20241006_144555.jpg
  inflating: /content/img/img/IMG_20241006_144641.jpg
  inflating: /content/img/img/IMG_20241006_144753.jpg
  inflating: /content/img/img/IMG_20241006_144902.jpg
  inflating: /content/img/img/IMG_20241006_144955.jpg
  inflating: /content/img/img/IMG_20241006_145056.jpg
  inflating: /content/img/img/IMG_20241006_145141.jpg
  inflating: /content/img/img/IMG_20241006_145250 - Copy.jpg
  inflating: /content/img/img/IMG_20241006_145250.jpg
  inflating: /content/img/img/IMG_20241006_145439.jpg
  inflating: /content/img/img/IMG_20241006_145636.jpg
  inflating: /content/img/img/IMG_20241006_145902.jpg
  inflating: /content/img/img/IMG_20241006_150012 - Copy.jpg
  inflating: /content/img/img/IMG_20241006_150012.jpg
  inflating: /content/img/img/IMG_20241006_150045.jpg
  inflating: /content/img/img/IMG_20241006_150130.jpg
  inflating: /content/img/img/IMG_20241006_150224.jpg
  inflating: /content/img/img/IMG_20241006_150321.jpg
  inflating: /content/img/img/IMG_20241006_150507.jpg
  inflating: /content/img/img/IMG_20241006_150800.jpg
  inflating: /content/img/img/IMG_20241006_150836.jpg
  inflating: /content/img/img/IMG_20241006_150902.jpg
  inflating: /content/img/img/IMG_20241006_150946.jpg
  inflating: /content/img/img/IMG_20241006_151057.jpg
  inflating: /content/img/img/IMG_20241006_151126.jpg
  inflating: /content/img/img/IMG_20241006_151222.jpg
  inflating: /content/img/img/IMG_20241006_151517.jpg
  inflating: /content/img/img/IMG_20241006_151556.jpg
  inflating: /content/img/img/IMG_20241006_151638.jpg
  inflating: /content/img/img/IMG_20241006_151719.jpg
  inflating: /content/img/img/IMG_20241006_151807.jpg
  inflating: /content/img/img/IMG_20241006_151901.jpg
  inflating: /content/img/img/IMG_20241006_152046.jpg
  inflating: /content/img/img/IMG_20241006_152156.jpg
  inflating: /content/img/img/IMG_20241006_152333.jpg
  inflating: /content/img/img/IMG_20241006_152453.jpg
  inflating: /content/img/img/IMG_20241006_152643.jpg
  inflating: /content/img/img/IMG_20241006_152937.jpg
  inflating: /content/img/img/IMG_20241006_153027.jpg
  inflating: /content/img/img/IMG_20241006_153118.jpg
  inflating: /content/img/img/IMG_20241006_153145.jpg
  inflating: /content/img/img/IMG_20241006_153240.jpg
  inflating: /content/img/img/IMG_20241006_153705.jpg
  inflating: /content/img/img/IMG_20241006_153802.jpg
  inflating: /content/img/img/IMG_20241006_153855.jpg
```

Load Dataset

```
df = pd.read_csv("/content/doctor_handwriting_labels.csv")

IMAGE_DIR = "/content/img/img"
IMG_WIDTH = 128
IMG_HEIGHT = 32
```

```
df.head()
```

	filename	label
0	.trashed-1730802496-IMG_20241006_151957 - Copy...	cefiget 40mg
1	IMG_20241006_141949 - Copy.jpg	tab azimax
2	IMG_20241006_142328 - Copy.jpg	tab toniflex
3	IMG_20241006_142533 - Copy.jpg	nims
4	IMG_20241006_142757 - Copy.jpg	dicloran

Character Set & Encoding

```
characters = sorted(list(set("".join(df['label']))))

char_to_num = layers.StringLookup(vocabulary=characters, mask_token=None)
num_to_char = layers.StringLookup(
    vocabulary=char_to_num.get_vocabulary(), invert=True
)

max_label_length = max(df['label'].str.len())

print("Characters:", characters)
print("Max label length:", max_label_length)
```

```
Characters: [' ', '-', '0', '1', '2', '4', '5', '6', '8', '9', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'i', 'k', 'l', 'm', 'n', '
Max label length: 17
```

Image Preprocessing Function

```
def load_image(img_path):
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
    img = img.astype(np.float32) / 255.0
    img = np.expand_dims(img, axis=-1)
    return img
```

Dataset Generator

```
def _load_image_from_path(img_path_tensor):

    img_path = img_path_tensor.numpy().decode('utf-8')
    return load_image(img_path)

def encode_sample(row):
    filename_tensor = row['filename']
    label_tensor = row['label']


    full_img_path_tensor = tf.strings.join([tf.constant(IMAGE_DIR), filename_tensor], separator='/')


    image = tf.py_function(
        func=_load_image_from_path,
        inp=[full_img_path_tensor],
        Tout=tf.float32
    ) # Added the missing closing parenthesis here


    image.set_shape([IMG_HEIGHT, IMG_WIDTH, 1])


    label = tf.strings.unicode_split(label_tensor, "UTF-8")
    label = char_to_num(label)


    padding = tf.zeros([max_label_length - tf.shape(label)[0]], dtype=tf.int64)
    padded_label = tf.concat([label, padding], axis=0)
    padded_label.set_shape([max_label_length])

    return {"image": image, "label": padded_label}

dataset = tf.data.Dataset.from_tensor_slices(dict(df))
dataset = dataset.map(encode_sample)

train_size = int(0.8 * len(df))
train_ds = dataset.take(train_size).batch(16)
val_ds = dataset.skip(train_size).batch(16)
```

CTC Loss Layer

```
class CTCLayer(layers.Layer):
    def call(self, y_true, y_pred):
        batch = tf.shape(y_true)[0]
        input_len = tf.shape(y_pred)[1]
        label_len = tf.shape(y_true)[1]
```

```
        input_len = input_len * tf.ones((batch, 1), dtype="int32")
        label_len = label_len * tf.ones((batch, 1), dtype="int32")

        loss = tf.keras.backend.ctc_batch_cost(
            y_true, y_pred, input_len, label_len
        )
        self.add_loss(loss)
        return y_pred
```

Build CRNN Model

```
image_input = layers.Input(shape=(IMG_HEIGHT, IMG_WIDTH, 1), name="image")
label_input = layers.Input(name="label", shape=(None,), dtype="int32")

x = layers.Conv2D(64, (3,3), activation="relu", padding="same")(image_input)
x = layers.MaxPooling2D((2,2))(x)

x = layers.Conv2D(128, (3,3), activation="relu", padding="same")(x)
x = layers.MaxPooling2D((2,2))(x)

x = layers.Reshape((IMG_WIDTH//4, (IMG_HEIGHT//4)*128))(x)

x = layers.Bidirectional(layers.LSTM(128, return_sequences=True))(x)
x = layers.Bidirectional(layers.LSTM(64, return_sequences=True))(x)

x = layers.Dense(len(char_to_num.get_vocabulary()) + 1, activation="softmax")(x)

output = CTCLayer()(label_input, x)

model = models.Model([image_input, label_input], output)
model.compile(optimizer="adam")
model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
image (InputLayer)	(None, 32, 128, 1)	0	-
conv2d (Conv2D)	(None, 32, 128, 64)	640	image[0][0]
max_pooling2d (MaxPooling2D)	(None, 16, 64, 64)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 16, 64, 128)	73,856	max_pooling2d[0]…
max_pooling2d_1 (MaxPooling2D)	(None, 8, 32, 128)	0	conv2d_1[0][0]
reshape (Reshape)	(None, 32, 1024)	0	max_pooling2d_1[…
bidirectional (Bidirectional)	(None, 32, 256)	1,180,672	reshape[0][0]
bidirectional_1 (Bidirectional)	(None, 32, 128)	164,352	bidirectional[0]…
label (InputLayer)	(None, None)	0	-
dense (Dense)	(None, 32, 34)	4,386	bidirectional_1[…
ctc_layer (CTCLayer)	(None, 32, 34)	0	label[0][0], dense[0][0]

Total params: 1,423,906 (5.43 MB)
Trainable params: 1,423,906 (5.43 MB)

Train the Model

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=30
)
```

```
Epoch 1/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 13s 860ms/step - loss: 1295.0210 - val_loss: 695.6924
Epoch 2/30
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 861ms/step - loss: 806.8473 - val_loss: 653.2346
Epoch 3/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 586ms/step - loss: 761.9086 - val_loss: 630.5881
Epoch 4/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 590ms/step - loss: 730.6512 - val_loss: 597.5006
Epoch 5/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 596ms/step - loss: 702.2557 - val_loss: 565.3440
Epoch 6/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 868ms/step - loss: 683.3994 - val_loss: 543.0826
Epoch 7/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 572ms/step - loss: 645.8685 - val_loss: 525.5404
Epoch 8/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 574ms/step - loss: 637.5002 - val_loss: 515.0687
Epoch 9/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 580ms/step - loss: 620.1044 - val_loss: 507.3278
Epoch 10/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 853ms/step - loss: 610.8254 - val_loss: 498.9034
Epoch 11/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 582ms/step - loss: 605.4977 - val_loss: 496.3874
Epoch 12/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 598ms/step - loss: 597.2238 - val_loss: 492.4692
Epoch 13/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 589ms/step - loss: 595.9341 - val_loss: 492.5531
Epoch 14/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 976ms/step - loss: 589.7499 - val_loss: 489.8741
Epoch 15/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 582ms/step - loss: 587.8453 - val_loss: 491.5516
Epoch 16/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 591ms/step - loss: 583.6562 - val_loss: 487.6514
Epoch 17/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 572ms/step - loss: 581.8669 - val_loss: 489.9617
Epoch 18/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 5s 1s/step - loss: 578.1882 - val_loss: 486.6003
Epoch 19/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 583ms/step - loss: 577.5259 - val_loss: 486.3463
Epoch 20/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 584ms/step - loss: 573.4189 - val_loss: 485.2082
Epoch 21/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 565ms/step - loss: 571.8505 - val_loss: 485.2748
Epoch 22/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 852ms/step - loss: 568.5587 - val_loss: 482.0485
Epoch 23/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 577ms/step - loss: 565.1389 - val_loss: 481.9866
Epoch 24/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 634ms/step - loss: 563.1674 - val_loss: 484.7459
Epoch 25/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 577ms/step - loss: 561.3445 - val_loss: 478.6747
Epoch 26/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 5s 995ms/step - loss: 558.5739 - val_loss: 478.3165
Epoch 27/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 580ms/step - loss: 555.2444 - val_loss: 475.7449
Epoch 28/30
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 601ms/step - loss: 551.9254 - val_loss: 476.1224
Epoch 29/30
```

Prediction Model

```
prediction_model = models.Model(
    image_input,
    model.layers[-2].output
)

def decode(pred):
    input_len = np.ones(pred.shape[0]) * pred.shape[1]
    decoded = tf.keras.backend.ctc_decode(pred, input_length=input_len)[0][0]
    texts = []
    for seq in decoded:
        text = "".join([num_to_char(c).numpy().decode() for c in seq if c != -1])
        texts.append(text)
    return texts
```

Test Prediction

```
sample_img = load_image(os.path.join(IMAGE_DIR, df.iloc[0]['filename']))
sample_img = np.expand_dims(sample_img, axis=0)

prediction = prediction_model.predict(sample_img)
print("Predicted Text:", decode(prediction))
print("Actual Text:", df.iloc[0]['label'])
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Predicted Text: ['ta [UNK]']
Actual Text: cefiget 40mg
```

XAI Parts

```
!pip install lime shap opencv-python
```

```
Requirement already satisfied: lime in /usr/local/lib/python3.12/dist-packages (0.2.0.1)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.50.0)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.6
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->l
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lim
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->l
```

Accuracy Evaluation

```
def evaluate_ocr(pred_model, dataset, df):
    correct_count = 0
    total_chars = 0
    correct_chars = 0

    for i, row in df.iterrows():
        img = load_image(os.path.join(IMAGE_DIR, row['filename']))
        img = np.expand_dims(img, axis=0)
        pred = pred_model.predict(img)
        pred_text = decode(pred)[0]

        true_text = row['label']

        # Exact match
        if pred_text == true_text:
            correct_count += 1

        # Character-level accuracy
        min_len = min(len(pred_text), len(true_text))
        for j in range(min_len):
            if pred_text[j] == true_text[j]:
                correct_chars += 1
        total_chars += len(true_text)

    exact_match_acc = correct_count / len(df)
    char_level_acc = correct_chars / total_chars

    return exact_match_acc, char_level_acc

exact_acc, char_acc = evaluate_ocr(prediction_model, val_ds, df)
print(f"Exact Match Accuracy: {exact_acc*100:.2f}%")
print(f"Character-level Accuracy: {char_acc*100:.2f}%")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 173ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 176ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 122ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 112ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 110ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 115ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 109ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 172ms/step
```

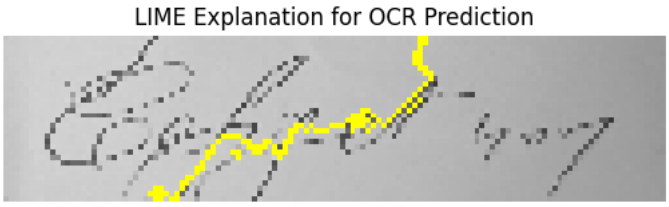
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 177ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 244ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 117ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 74ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 81ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 67ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 67ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 67ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 111ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 95ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 97ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 97ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 100ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 103ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 104ms/step
```

Explainable AI with LIME

```
from lime import lime_image
from skimage.segmentation import mark_boundaries
from skimage.color import gray2rgb
import matplotlib.pyplot as plt
import cv2
import tensorflow as tf
import numpy as np


sample_img_rgb = gray2rgb(sample_img[:,:,0])

explainer = lime_image.LimeImageExplainer()

def ocr_predict(images_input_from_lime):
    processed_images = []
    for img in images_input_from_lime:
        # Convert RGB to grayscale (your model expects 1 channel)
        gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
        resized = cv2.resize(gray, (IMG_WIDTH, IMG_HEIGHT))
        img_4d = resized[:, :, np.newaxis].astype(np.float32)/255.0
        processed_images.append(img_4d)

    batch_images = np.array(processed_images, dtype=np.float32)
    tf_images = tf.convert_to_tensor(batch_images, dtype=tf.float32)

    preds = prediction_model.predict(tf_images, verbose=0)


    flattened = np.array([np.max(p, axis=-1) for p in preds])
    return flattened

# Explain the instance
explanation = explainer.explain_instance(
```

```
    sample_img_rgb,
    ocr_predict,
    top_labels=1,
    hide_color=0,
    num_samples=1000
)

temp, mask = explanation.get_image_and_mask(
    explanation.top_labels[0],
    positive_only=True,
    num_features=10,
    hide_rest=False
)

plt.imshow(mark_boundaries(temp, mask))
plt.axis('off')
plt.title("LIME Explanation for OCR Prediction")
plt.show()
```

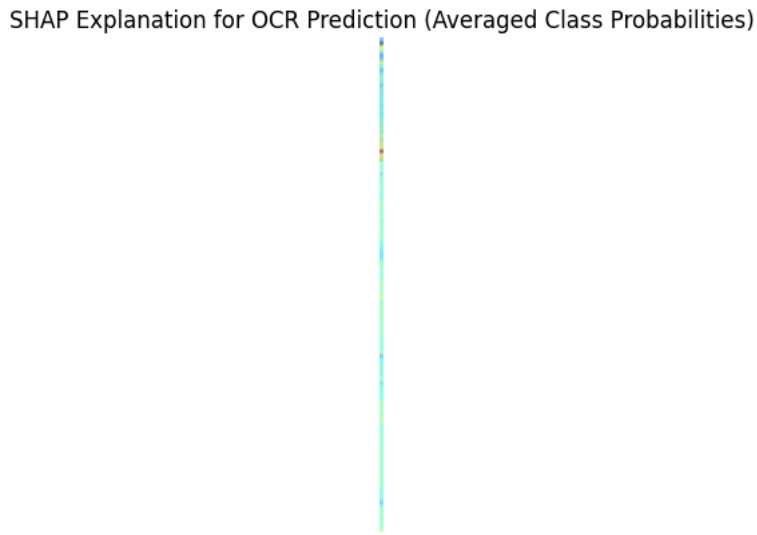
```
100%                                                  1000/1000 [00:21<00:00, 49.90it/s]
```



Explainable AI with SHAP

```
import shap

# Create a SHAP-compatible model by taking the Global Average Pooling of the prediction_model's output
# The prediction_model's output has shape (None, IMG_WIDTH//4, num_classes)
# We want to reduce it to (None, num_classes) for SHAP GradientExplainer
shap_model_input = prediction_model.input
shap_model_output = layers.GlobalAveragePooling1D()(prediction_model.output)
shap_compatible_model = models.Model(inputs=shap_model_input, outputs=shap_model_output)


background = np.stack([load_image(os.path.join(IMAGE_DIR, f))[:,:,0] for f in df['filename'][:20]], axis=0)
background = np.expand_dims(background, -1)

# Use the newly created shap_compatible_model with GradientExplainer
e = shap.GradientExplainer(shap_compatible_model, background)

sample = np.expand_dims(sample_img, 0)
shap_values = e.shap_values(sample)

plt.imshow(sample_img[:,:,0], cmap='gray')
# shap_values[0] corresponds to the SHAP values for the first output class (first character in vocabulary)
# shap_values[0][0] corresponds to the first sample
plt.imshow(shap_values[0][0][:,:,0], cmap='jet', alpha=0.5)
plt.axis('off')
plt.title("SHAP Explanation for OCR Prediction (Averaged Class Probabilities)")
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: The structure of `inputs` doesn't m
Expected: image
Received: inputs=['Tensor(shape=(1, 32, 128, 1))']
  warnings.warn(msg)
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: The structure of `inputs` doesn't m
Expected: image
Received: inputs=['Tensor(shape=(50, 32, 128, 1))']
  warnings.warn(msg)
```



Prediction + XAI

```
sample_idx = 0
```